

Componentes en JSF



Componentes en JSF

1. ¿Qué son los componentes en JSF?

JavaServer Faces (JSF) es un framework de Java para el desarrollo de aplicaciones web basadas en componentes. Los **componentes** en JSF representan elementos de la interfaz de usuario (UI), como botones, formularios, tablas, entre otros.

Cada componente en JSF tiene un propósito específico y permite asociar eventos y datos, simplificando la interacción entre el frontend (interfaz) y el backend (lógica del servidor).

Un **componente JSF** es una unidad reutilizable de la interfaz gráfica que:

- Se representa mediante una etiqueta en el archivo XHTML.
- Tiene un estado y comportamiento controlado por el servidor.
- Puede estar vinculado a propiedades de un bean administrado.

Ejemplo de componentes en JSF:

- `<h:outputText>`: Muestra texto.
- `<h:inputText>`: Permite al usuario ingresar texto.
- `<h:commandButton>`: Botón que ejecuta una acción en el servidor.

2. Sintaxis Básica de un Componente

Los componentes en JSF se definen dentro de un archivo **XHTML** utilizando etiquetas con el prefijo **h:** (por "HTML Render Kit").

Estructura básica de un componente:

```
<h:componente atributo="valor" />
```

Por ejemplo:

```
<h:outputText value="¡Hola, Mundo!" />
```

Este componente renderiza texto en la página web.

Ejemplo Completo

1. Crear el proyecto en NetBeans

1. Abre NetBeans y selecciona **File > New Project**.
2. En **Maven**, elige **Jakarta EE with Maven** y haz clic en **Next**.
3. Configura el nombre del proyecto: ComponentesJSF
4. Selecciona **Jakarta EE** como el perfil.
5. Elige **GlassFish Server** como servidor de aplicaciones.
6. Haz clic en **Finish**.

2. Crear los recursos paso a paso

Clase de modelo: Candidato.java

Esta clase define los datos que manejará el formulario, como nombre, apellido, y salario deseado.

```
package beans.model;

import jakarta.enterprise.context.SessionScoped;
import jakarta.inject.Named;
import java.io.Serializable;
import java.util.Date;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;

@Named("candidato")
@SessionScoped
public class Candidato implements Serializable {
```

```
private Logger log = LoggerFactory.getLogger(Candidato.class);

private String nombre;
private String apellido;
private int salarioDeseado;
private Date fechaNacimiento;
private String codigoPostal;
private String ciudad;
private String comentario;
private int coloniaId;

public Candidato(){
    log.info("Creando el objeto Candidato");
}

public String getNombre() {
    return nombre;
}

public void setNombre(String nombre) {
    this.nombre = nombre;
    log.info("Modificando la propiedad de nombre:" + this.nombre);
}

public String getApellido() {
    return apellido;
}

public void setApellido(String apellido) {
    this.apellido = apellido;
    log.info("Modificando la propiedad de apellido: " + this.apellido);
}

public int getSalarioDeseado() {
    return salarioDeseado;
}

public void setSalarioDeseado(int salarioDeseado) {
    this.salarioDeseado = salarioDeseado;
    log.info("Modificando la propiedad de sueldoDeseado: " + this.salarioDeseado);
}

public Date getFechaNacimiento() {
    return fechaNacimiento;
}
```

```
}

public void setFechaNacimiento(Date fechaNacimiento) {
    this.fechaNacimiento = fechaNacimiento;
}

public String getCodigoPostal() {
    return codigoPostal;
}

public void setCodigoPostal(String codigoPostal) {
    this.codigoPostal = codigoPostal;
}

public String getCiudad() {
    return ciudad;
}

public void setCiudad(String ciudad) {
    this.ciudad = ciudad;
}

public String getComentario() {
    return comentario;
}

public void setComentario(String comentario) {
    this.comentario = comentario;
}

public int getColoniaId() {
    return coloniaId;
}

public void setColoniaId(int coloniaId) {
    this.coloniaId = coloniaId;
}
}
```

Backing Bean: vacanteForm.java

```
package beans.backing;

import beans.model.Candidato;
```

```
import jakarta.enterprise.context.RequestScoped;
import jakarta.faces.application.FacesMessage;
import jakarta.faces.component.UIInput;
import jakarta.faces.component.UIViewRoot;
import jakarta.faces.context.FacesContext;
import jakarta.faces.event.ActionEvent;
import jakarta.faces.event.ValueChangeEvent;
import jakarta.inject.Inject;
import jakarta.inject.Named;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;

@Named
@RequestScoped
public class VacanteForm {

    private Logger log = LoggerFactory.getLogger(VacanteForm.class);

    private boolean comentarioEnviado;

    @Inject
    private Candidato candidato;

    public VacanteForm() {
        log.info("Creando el objeto VacanteForm");
    }

    public void setCandidato(Candidato candidato) {
        this.candidato = candidato;
    }

    public String enviar() {
        if ("Juan".equals(candidato.getNombre())
            && "Perez".equals(candidato.getApellido())) {
            String msg = "Gracias Juan Perez, te has registrado correctamente.";
            FacesMessage facesMessage =
                new FacesMessage(FacesMessage.SEVERITY_ERROR, msg, null);
            FacesContext.getCurrentInstance().addMessage(null, facesMessage);
            log.info("Usuario registrado correctamente");
            return "index"; // Redirige a la misma página
        }
        log.info("Datos de usuario inválidos");
        return "fallo"; // Redirige a una página de éxito
    }
}
```

```
public void codigoPostalListener(ValueChangeEvent valueChangeEvent) {
    FacesContext facesContext = FacesContext.getCurrentInstance();
    UIViewRoot uiViewRoot = facesContext.getViewRoot();
    String nuevoCodigoPostal = (String) valueChangeEvent.getNewValue();
    if ("03810".equals(nuevoCodigoPostal)) {
        UIInput coloniaIdInputText =
            (UIInput) uiViewRoot.findComponent("vacanteForm:coloniaId");
        int coloniaId = 1;
        coloniaIdInputText.setValue(coloniaId);
        coloniaIdInputText.setSubmittedValue(coloniaId);

        UIInput ciudadInputText =
            (UIInput) uiViewRoot.findComponent("vacanteForm:ciudad");
        String nuevaCiudad = "Ciudad de Mexico";
        ciudadInputText.setValue(nuevaCiudad);
        ciudadInputText.setSubmittedValue(nuevaCiudad);

        facesContext.renderResponse();
    }
    else{
        // Reseteamos los valores
        UIInput coloniaIdInputText =
            (UIInput) uiViewRoot.findComponent("vacanteForm:coloniaId");
        int coloniaId = 0;
        coloniaIdInputText.setValue(coloniaId);
        coloniaIdInputText.setSubmittedValue(coloniaId);
        UIInput ciudadInputText =
            (UIInput) uiViewRoot.findComponent("vacanteForm:ciudad");
        String nuevaCiudad = "";
        ciudadInputText.setValue(nuevaCiudad);
        ciudadInputText.setSubmittedValue(nuevaCiudad);

        facesContext.renderResponse();
    }
}

public void ocultarComentario(ActionEvent actionEvent){
    this.comentarioEnviado = !this.comentarioEnviado;
}

public boolean isComentarioEnviado() {
    return comentarioEnviado;
}

public void setComentarioEnviado(boolean comentarioEnviado) {
```

```

        this.comentarioEnviado = comentarioEnviado;
    }
}

```

Archivos de internacionalización

1. Crea una carpeta llamada `resources` dentro de `src/main/webapp/WEB-INF`.
2. Agrega los archivos `mensajes_en.properties` y `mensajes_es.properties`:
 - o **`mensajes_es.properties`**:

```

vacanteForm.nombre=Nombre
vacanteForm.apellido=Apellido
vacanteForm.salarioDeseado=Salario Deseado
vacanteForm.fechaNacimiento=Fecha de Nacimiento
vacanteForm.codigoPostal=Codigo Postal
vacanteForm.coloniaId=Colonia
vacanteForm.seleccione=--Seleccione--
vacanteForm.ciudad=Ciudad
vacanteForm.comentario=Comentario
vacanteForm.mostrar=Mostrar
vacanteForm.ocultar=Ocultar
vacanteForm.enviar=Enviar

```

- o **`mensajes_en.properties`**:

```

vacanteForm.nombre=Name
vacanteForm.apellido=Last Name
vacanteForm.salarioDeseado=Desired Salary
vacanteForm.fechaNacimiento=Birthdate
vacanteForm.codigoPostal=Zip Code
vacanteForm.coloniaId=Suburb
vacanteForm.seleccione=--Select--
vacanteForm.ciudad=City
vacanteForm.comentario=Annotation
vacanteForm.mostrar=Show
vacanteForm.ocultar=Hide
vacanteForm.enviar=Submit

```

Página de inicio: `index.xhtml`

Esta página define el formulario de registro con componentes JSF:

1. Crea el archivo `index.xhtml` en `src/main/webapp`.

```
<!DOCTYPE html>
```

```

<html xmlns="http://www.w3.org/1999/xhtml"
      xmlns:h="jakarta.faces.html"
      xmlns:f="jakarta.faces.core">
  <h:head>
    <title>Bolsa de Trabajo V7</title>
  </h:head>
  <h:body>
    <h:form id="vacanteForm">
      <h:messages globalOnly="true" ></h:messages>

      <table>
        <tr>
          <td><h:outputLabel for="nombre"
            value="#{msgs['vacanteForm.nombre']}"></h:outputLabel></td>
          <td><h:inputText id="nombre" required="true"
            value="#{candidato.nombre}"></h:inputText></td>
          <td><h:message for="nombre"></h:message></td>
        </tr>
        <tr>
          <td><h:outputLabel for="apellido"
            value="#{msgs['vacanteForm.apellido']}"></h:outputLabel></td>
          <td><h:inputText id="apellido" required="true"
            value="#{candidato.apellido}"></h:inputText></td>
          <td><h:message for="apellido"></h:message></td>
        </tr>
        <tr>
          <td><h:outputLabel for="salarioDeseado"
            value="#{msgs['vacanteForm.salarioDeseado']}"></td>
          <td><h:inputText id="salarioDeseado" required="true"
            value="#{candidato.salarioDeseado}"
            <f:validateLongRange minimum="500" maximum="5000"/>
            </h:inputText>
          </td>
          <td><h:message for="salarioDeseado"></h:message></td>
        </tr>
        <tr>
          <td><h:outputLabel for="fechaNacimiento"
            value="#{msgs['vacanteForm.fechaNacimiento']}"></td>
          <td><h:inputText id="fechaNacimiento" required="true"
            value="#{candidato.fechaNacimiento}"
            <f:convertDateTime pattern="dd/MM/yyyy"></f:convertDateTime>
            </h:inputText>
          </td>
          <td><h:message for="fechaNacimiento"></h:message></td>
        </tr>
      </table>
    </h:form>
  </h:body>
</html>

```

```

<tr>
  <td><h:outputLabel for="codigoPostal"
    value="#{msgs['vacanteForm.codigoPostal']}" ></h:outputLabel></td>
  <td><h:inputText id="codigoPostal" immediate="true"
    onchange="this.form.submit()"
    required="true" value="#{candidato.codigoPostal}"

    valueChangeListener="#{vacanteForm.codigoPostallistener}">
  </h:inputText></td>
  <td><h:message for="codigoPostal"></h:message></td>
</tr>
<tr>
  <td><h:outputLabel for="coloniaId"
    value="#{msgs['vacanteForm.coloniaId']}" ></h:outputLabel></td>
  <td>
    <h:selectOneMenu id="coloniaId" required="true"
      value="#{candidato.coloniaId}">
      <f:selectItem itemLabel="#{msgs['vacanteForm.seleccione']}"
        itemValue="" />
      <f:selectItem itemLabel="Napoles" itemValue="1" />
      <f:selectItem itemLabel="Polanco" itemValue="2" />
      <f:selectItem itemLabel="Del Valle Centro" itemValue="3" />
    </h:selectOneMenu>
  </td>
  <td><h:message for="coloniaId"></h:message></td>
</tr>
<tr>
  <td><h:outputLabel for="ciudad"
    value="#{msgs['vacanteForm.ciudad']}" ></h:outputLabel></td>
  <td><h:inputText id="ciudad" required="true"
    value="#{candidato.ciudad}"></h:inputText></td>
  <td><h:message for="ciudad"></h:message></td>
</tr>
</table>
<table>
  <tr>
    <td>
      <h:commandLink actionListener="#{vacanteForm.ocultarComentario}"
        immediate="true"
        rendered="#{!vacanteForm.comentarioEnviado}"
        value="#{msgs['vacanteForm.mostrar']}" />
      <h:commandLink actionListener="#{vacanteForm.ocultarComentario}"
        immediate="true"
        rendered="#{vacanteForm.comentarioEnviado}"
        value="#{msgs['vacanteForm.ocultar']}" />
    </td>
  </tr>
</table>

```

```

        </td>
    </tr>
</table>
<h:panelGroup rendered="#{vacanteForm.comentarioEnviado}">
    <table>
        <tr>
            <td><h:outputLabel for="comentario"
                value="#{msgs['vacanteForm.comentario']}" /></td>

            </tr>
            <tr>
                <td><h:inputTextarea id="comentario" cols="50" rows="10"
                    value="#{candidato.comentario}" /></td>

            </tr>
        </table>
    </h:panelGroup>
    <h:commandButton action="#{vacanteForm.enviar}"
        value="#{msgs['vacanteForm.enviar']}" />
</h:form>
</h:body>
</html>

```

3. Implementación de mejores prácticas

1. **Uso de Logging:** Se utiliza SLF4J para registrar información útil.
2. **Inyección de dependencias con @Inject:** Simplifica el manejo del ciclo de vida de los beans.
3. **Validaciones y conversores estándar de JSF:**
 - Validación de rango en el salario.
 - Conversión de fechas con `f:convertDateTime`.
4. **Internacionalización:** Los mensajes se configuran en diferentes idiomas.
5. **Separación de responsabilidades:** La lógica se encapsula en el bean `VacanteForm`, manteniendo el modelo (`Candidato`) simple.

4. Despliegue de la aplicación

1. En NetBeans, selecciona **Run > Clean and Build Project**.
2. Una vez compilado, selecciona **Run > Run Project**.
3. Accede a la aplicación desplegada

5. Explicación de los componentes utilizados

1. **Formulario (`h:form`):** Agrupa todos los componentes relacionados con los datos del candidato.
2. **Entrada de datos (`h:inputText`):** Permite capturar información como nombre, apellido, salario y fecha.

3. **SelectOneMenu** (`h:selectOneMenu`): Muestra opciones de colonias basadas en el código postal.
4. **Mensajes** (`h:messages` y `h:message`): Muestra errores de validación o confirmaciones.
5. **Conversor** (`f:convertDateTime`): Convierte el formato de la fecha a dd/MM/yyyy.
6. **Validador** (`f:validateLongRange`): Valida que el salario deseado esté dentro del rango permitido.

Conclusión

En esta lección, creamos una aplicación que utiliza varios componentes de JSF. Además de aplicar las mejores prácticas, como el uso de logging, internacionalización, y separación de responsabilidades. La aplicación sirve como base para proyectos más avanzados en el desarrollo web con Java y JSF.

¡Sigue adelante con tu aprendizaje 🚀, el esfuerzo vale la pena!

Saludos!

Ing. Ubaldo Acosta

Fundador de [GlobalMentoring.com.mx](https://www.globalmentoring.com.mx)